Chapter 14

A SPLITTING ALGORITHM

The tools for boundary modeling presented in the preceding chapters form the basis of yet higher-level tools. This chapter describes an algorithm for *splitting* a solid with a plane. While being a useful modeling tool by itself, it also forms a good introduction to the still more advanced subject of Boolean set operations, the topic of the next chapter.

14.1 THE PROBLEM

We shall consider an algorithm for solving the *splitting problem*: Given a solid S, and a splitting plane SP, calculate two solids *Above* and *Below* that represent the parts of S that lie on the positive and on the negative side of SP, respectively. If S lies completely on one side of SP, the solid representing the other side is empty.

Observe that this problem definition requires that the algorithm is *closed*: its results must be "solids" themselves, and hence the algorithm (and any other algorithms that work for solids) must be repeatedly applicable to its results. This means that while a splitting algorithm certainly would be useful for generating the "section views" often encountered in engineering drawings (see Figure 14.1), it is not restricted to mere visualization, but can be used as a real modeling tool.

More precisely, we shall assume that S is in all respects a well-formed solid represented with a correct half-edge data structure. In particular, S must be nonself-intersecting, and all of its faces, edges, and vertices must have correct geometries (up to reasonable floating-point precision).

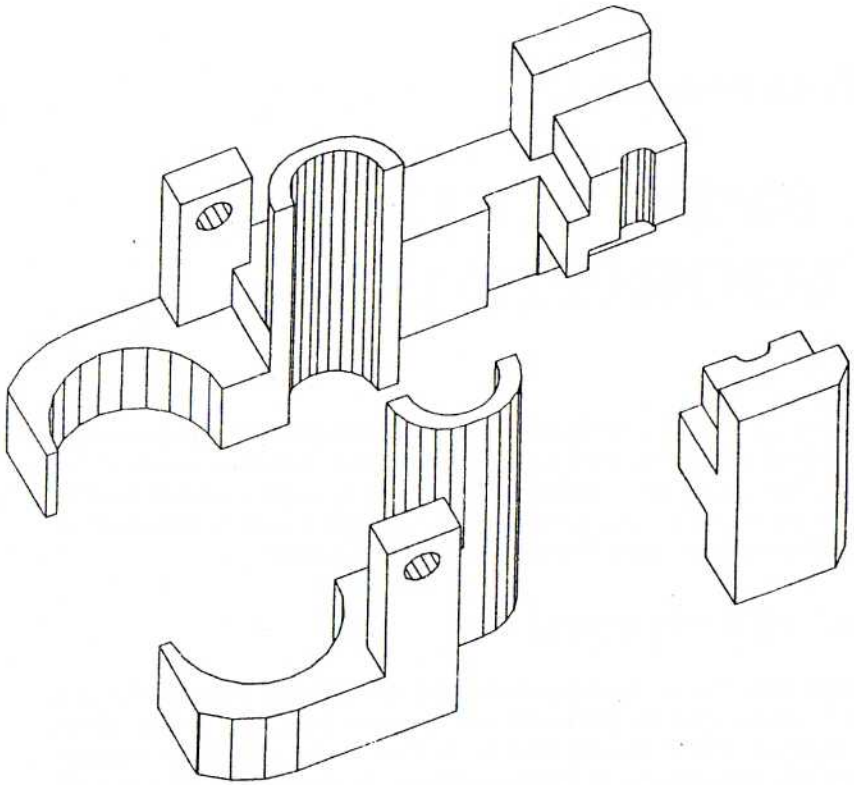


Figure 14.1 A split solid.

If so, we require that the results of a splitting algorithm have the same correctness characteristics.

To satisfy the problem statement, a splitting algorithm must be *general*, i.e., it must work correctly for all possible geometric configurations of solids and planes. The algorithm should also be *robust*, i.e., capable of working correctly even if small numerical inaccuracies are present in the original solid S.

In the following, we shall develop a splitting algorithm with these characteristics. In particular, the algorithm will be completely general up to the limits permitted by computational accuracy.

14.2 OVERVIEW

Let us first outline the splitting algorithm at a verbal level, and introduce some terminology that will be used also in the next chapter.

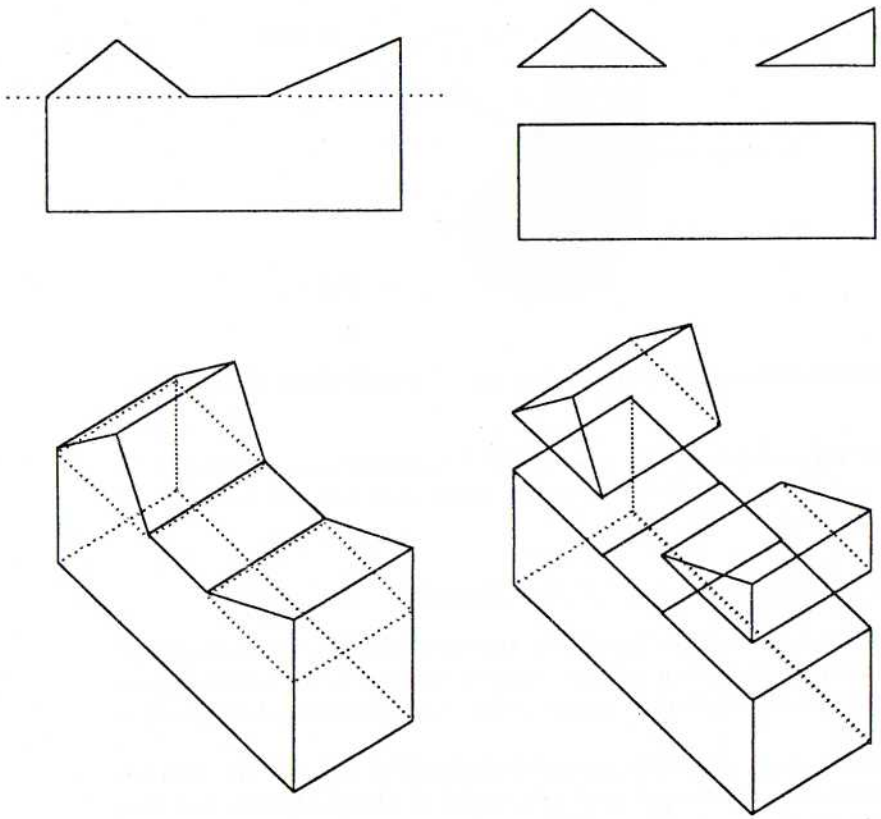
Obviously, if the solid S to be split and the splitting plane SP intersect at all, they do so along one or more cross section polygons or just *section polygons*. For instance, the object of Figure 14.1 has five section polygons.

The splitting algorithm is expected to divide S along the section polygon(s) in the two parts *Above* and *Below*. This can be achieved by inserting a pair of *section faces* along each section polygon, one in the part *Above* and the other in the part *Below*. (In engineering drawings, section faces are usually shown crosshatched.)

In the following, we shall outline this computation with help of the sample case shown in two views in Figure 14.2. In 14.2(a), the section polygons are drawn in dashed lines; 14.2(b) shows the expected result. Observe that the part *Above* in this case is a disconnected solid, which is a perfectly acceptable result of the algorithm. Observe also that several edges and vertices of the solid are coplanar with the splitting plane. (That the top surface of *Below* consists of three coplanar faces is an artifact of the splitting algorithm to be discussed.)

14.2.1 Reduction Step

Our plan for the splitting algorithm is to *reduce* the overall problem into a collection of subproblems that can be handled independently of each other. Because we aim at a simple solution that does not involve a large case analysis, we shall select the most primitive kind of a subproblem possible, namely the processing a vertex of the solid S lying on the splitting plane SP. For instance, instead of processing an edge lying on SP directly as one



(a) (b)

Figure 14.2 A sample splitting problem.

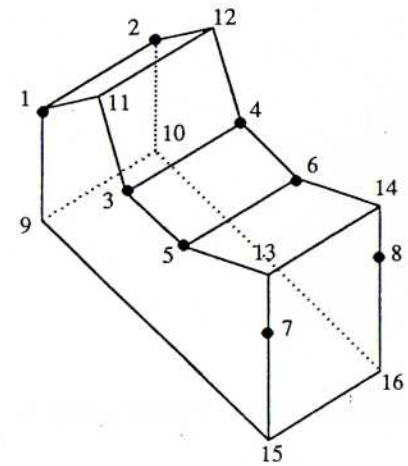


Figure 14.3 Set of coplanar vertices for the sample splitting problem.

particular case of some case analysis, we handle it by processing its two vertices appropriately.

This reduction is the task of the first step of the algorithm. It can be described as follows:

1. Locate all edges of S that intersect SP properly, i.e., whose vertices lie in different sides of SP. Subdivide each such edge with a new vertex at its intersection point with SP.

2. Locate all vertices of S that lie on SP, and store them for further processing. If no such vertices are found, we are done: the solid does not intersect SP at all. Of course, the resulting set of vertices includes at least all vertices inserted during step 1.

The set of vertices generated in the reduction step of the sample splitting problem of Figure 14.2 is shown in Figure 14.3. For reference, these vertices are labeled with integers 1–8. Labels 9–16 denote the other vertices of S. Observe that vertices 7 and 8 were inserted into S by subdividing an edge at its intersection point with SP.

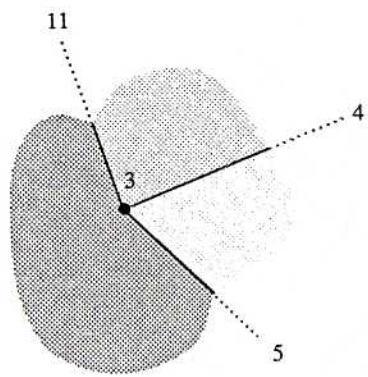


Figure 14.4 Vertex neighborhood of vertex 3.

14.2.2 Vertex Neighborhood Classification

By virtue of the reduction step, we now have only one kind of a basic problem to worry about, namely the processing of a vertex lying on *SP*. Of course, this computation must be general enough to cover all possible situations.

To achieve the desired generality with a simple algorithm, is is again useful to split the processing into more primitive subproblems. Let us introduce the term *vertex neighborhood* to denote the ordered cycle of edges and faces around a vertex. For instance, the vertex neighborhood of vertex 3 of the sample problem is depicted in Figure 14.4. This neighborhood consists of the (half-)edges (3,11), (3,4) and (3,5); the vertices 11, 4, and 5 are termed the *final vertices* of the edges of the vertex neighborhood, and vertex 3 is the *base vertex* of the neighborhood. We shall use the term *sector* to denote the part of a face immediately adjacent to a vertex; in Figure 14.4, the neighborhood includes sectors (11,3,4), (4,3,5), and (5,3,11).

To solve the splitting problem, each coplanar vertex will be processed with an algorithm called here the *vertex neighborhood classifier*. The classifier is expected to locate all sectors that intersect *SP*, and to subdivide the neighborhood into parts that are wholly on one side of *SP*.

The subdivision of a neighborhood is effected by inserting new edges of zero length into it; these *null edges* will be removed in later steps of the splitting algorithm. For instance, the result of the processing of vertex 3 is

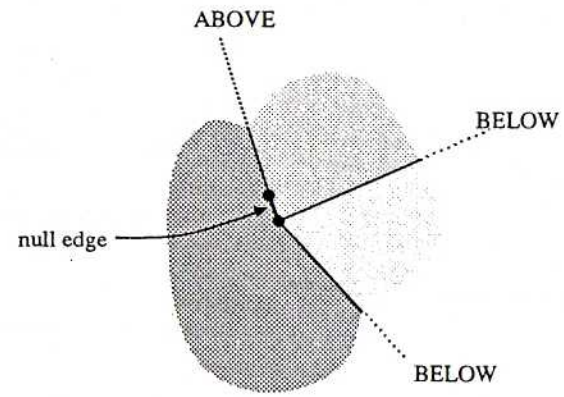


Figure 14.5 Result of vertex neighborhood classification of vertex 3.

shown in Figure 14.5. In this case, the null edge separates the edge (3,11) (which will appear in *Above*) from the edges (3,4) and (3,5) (which will appear in *Below*).

14.2.3 Computation of the Result

After all coplanar vertices have been processed with the vertex neighborhood classifier, the section polygons can be constructed by combining the vertices of a null edges in a proper order, and removing the null edges themselves.

For the sample case, this process is depicted in Figure 14.6. Observe that we can choose the topological orientation of all null edges so that they will be positively oriented from *Below* to *Above*. As will be seen, this leads to a relatively simple iterative algorithm that marches around a section polygon, and combines null edges with each other. After the joining step, it is straightforward to insert the section faces, and to rip *Above* and *Below* apart.

14.3 OUTLINE OF THE ALGORITHM

An outline of the splitting algorithm is given in Program 14.1. In addition to the half-edge data structure, the algorithm uses three arrays for representing the "set of ON-vertices" *soov*, the "set of null edges" *sone*, and the "set of null faces" *sonf* that will be used to store the section faces. These

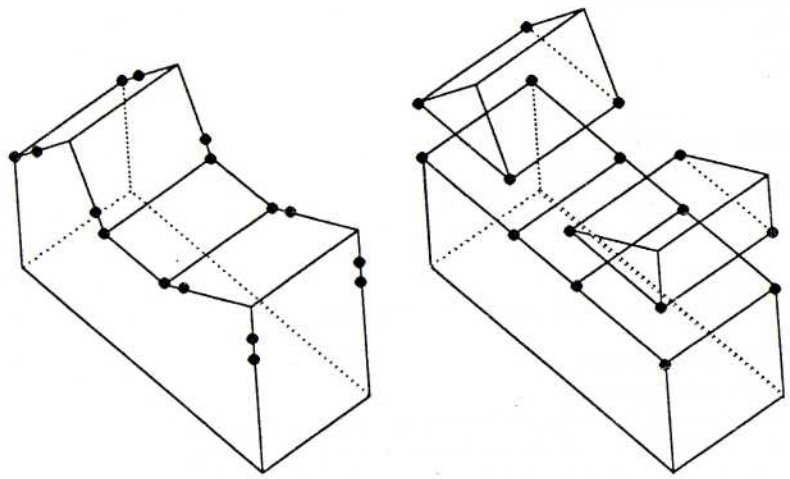


Figure 14.6 Computation of the result.

arrays are assumed to be global information available to all parts of the splitting algorithm.

The splitting plane SP is represented simply by a vector whose components give the four coefficients of the plane equation. For initialization, the algorithm evaluates all face equations of the solid, and sets up the global variables *maxv* and *maxf* for generating unique identifiers in later parts. The remaining procedures will be elaborated in following sections.

14.4 REDUCTION STEP

Let us first consider the procedure **splitgenerate** which is responsible for generating the set of coplanar vertices *soov* for the later steps. In particular, it must be capable of subdividing all edges of S that intersect SP at their intersection points.

Obviously, an edge e of S intersects the splitting plane SP if and only if its end points p_1 and p_2 lie on opposite sides of the plane. Hence **splitgenerate** can work by evaluating the signed distances d_1 and d_2 of p_1 and p_2. If either distance is (approximately) zero, the corresponding vertex is on SP, and can immediately be inserted to the result set. Otherwise, if the signs of the distances differ, e must intersect SP, and the point


```
Vertex  *soov[MAXONVERTICES];
int     nvtx;
Edge    *sone[MAXNULLEDGES];
int     nedg;
Face    *sonf[MAXNULLFACES*2];
int     nfac;

void    split(S, SP, Above, Below)
Solid   *S, **Above, **Below;
vector  SP;
{
        Face    *f;

        for(f = S->sfaces; f != NIL; f = f->nextf)
                faceeq(f->flout, f->feq);
        getmaxnames(S);
        splitgenerate(S, SP);
        splitclassify(SP);
        if(nedg == 0)
        {
                printf("split: no intersections found\n");
                return;
        }
        splitconnect();
        splitfinish(S, Above, Below);
}
```

Program 14.1 Outline of the splitting algorithm.

of intersection p_{int} can be calculated as

$$p_{int} = p_1 + \frac{d_1}{d_1 - d_2}(p_2 - p_1).$$

Procedure **splitgenerate** is implemented in Program 14.2. The algorithm scans all edges of s and calculates the distances *d1* and *d2* by means of the procedure **dist** not elaborated here. If the distances are nonzero and of different sign, the edge is subdivided, and the new vertex inserted to the result set. Otherwise, if either of the distances is zero, the correponding vertex is stored immediately. Of course, the comparisons with zero must be done with the procedure **comp**, which was introduced in Chapter 13.

Obviously, the algorithm will encounter each coplanar vertex several times. It could be made more efficient by calculating the signed distance of each vertex just once, and storing these data into an auxiliary array.

14.5 VERTEX NEIGHBORHOOD CLASSIFIER

Next, let us consider the implementation of the vertex neighborhood classifier **splitclassify**.

14.5.1 Reclassification Rules

The classifier must be capable of treating properly all kinds of vertex neighborhoods, including ones with sectors or edges that are coplanar with the splitting plane *SP*. Clearly, with all coplanar entities, the classifier will have to decide whether they should appear in the part *Above* or the part *Below* in the final result. We view this as a *reclassification* problem: entities on *SP* must be reclassified either as lying above or below *SP* according to certain rules which will guarantee the correctness of the overall result.

Fortunately, such a set of rules is not difficult to come by. Consider first Figure 14.7 which depicts a two-dimensional projection of two cases where a face of the solid S lies on the splitting plane *SP*. Obviously, each of the vertices of those faces will have one coplanar sector. In the case on the left, the coplanar face is expected to end up in the part *Below*, and on the right, the part *Above*. This can be achieved if the coplanar sectors and their edges are reclassified "to the side of the material."

We may also encounter coplanar edges that do not belong to any coplanar sectors, such as in the case of Figure 14.8. In the case depicted, the result of the sectioning cannot be represented as a 2-manifold, and we shall have to decide whether to treat the part *Above* as a single self-intersecting


```
void      splitgenerate(S, SP)
Solid     *S;
vector    SP;
{
     Edge          *e;
     HalfEdge      *he;
     Vertex        *v1, *v2;
     double        d1, d2, t, x, y, z;
     int           s1, s2;

     nvtx = 0;
     for(e = S->sedges; e != NIL; e = e->nexte)
     {
          v1 = e->he1->vtx;
          v2 = e->he2->vtx;
          s1 = comp((d1 = dist(v1->vcoord, SP)), 0.0, EPS);
          s2 = comp((d2 = dist(v2->vcoord, SP)), 0.0, EPS);
          if(s1 == -1 && s2 == 1 || s1 == 1 && s2 == -1)
          {
               t = d1 / (d1 - d2);
               x = v1->vcoord[0] + t*(v2->vcoord[0]-v1->vcoord[0]);
               y = v1->vcoord[1] + t*(v2->vcoord[1]-v1->vcoord[1]);
               z = v1->vcoord[2] + t*(v2->vcoord[2]-v1->vcoord[2]);
               lmev(e->he1, (he = e->he2->nxt), ++maxv, x, y, z);
               addsoov(he->prv->vtx);
          }
          else
          {
               if(s1 == 0) addsoov(v1);
               if(s2 == 0) addsoov(v2);
          }
     }
}

void      addsoov(v)
Vertex    *v;
{
     int       i;
     for(i=0; i<nvtx; i++) if(soov[i] == v) return;
     soov[nvtx++] = v;
}
```

Program 14.2 Reduction step.

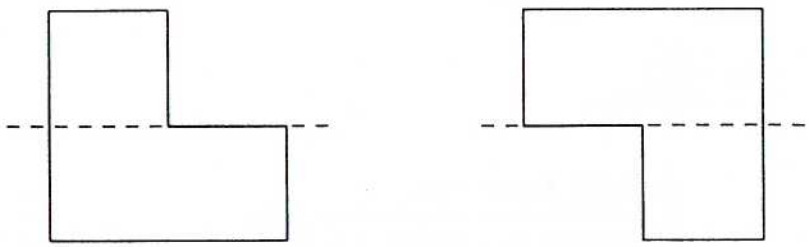


Figure 14.7 Treatement of coplanar sectors.

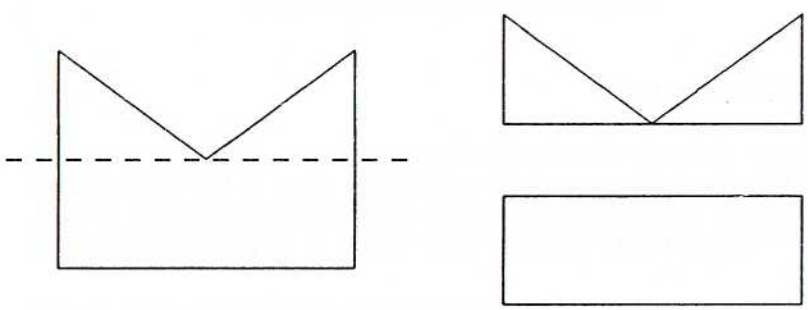


Figure 14.8 Treatment of coplanar edges.

object or two touching wedges. We prefer the latter; this result is achieved if the coplanar edge is treated as lying below *SP*.

This leads us to the following outline for the vertex neighborhood classification:

1. Classify each edge of the neighborhood according to whether its final vertex lies above, on, or below *SP*. Tag the edge with the corresponding label ABOVE, ON, or BELOW.

2. Consider all labeled edges in their cyclic order around the vertex, and reclassify all edges tagged with ON by applying the following rules in the order given:

   (a) For each coplanar sector, reclassify the edges of the sector as lying "on the side of the material," i.e., if the face normal of the face of the sector is colinear with the normal of *SP*, the edges are relassified as BELOW, and otherwise ABOVE.

   (b) After applying the previous rule, ON-edges may appear in only four kinds of consecutive edge arrangements, namely

   ABOVE-ON-ABOVE
   ABOVE-ON-BELOW
   BELOW-ON-BELOW
   BELOW-ON-ABOVE.

   In these cases, the ON-edge is reclassified as BELOW, BELOW, ABOVE, and BELOW, respectively.

Observe that rule (b) is designed so that nonmanifold results will be represented as disconnected models.

14.5.2 Implementation of the Classifier

The overall flow of the classifier is given in Program 14.3. The neighborhood can be conveniently represented as terms of an array of structures that gives one halfedge of the sector and a classification code. Although not explicitly shown, we assume the array is global information accessible to all routines of this section.

The classifier simply loops through all coplanar vertices, and calls three procedures described in the sequel that implement the classification rules. The procedure `insertnulledges`, to be described in Section 14.6, examines the final classification, and inserts the appropriate null edges so as to divide the neighborhood.

```
# define ABOVE 1
# define BELOW -1
# define ON 0

struct
{
        HalfEdge        *sector;
        int             cl;
} nbr[MAXEDGESFORVERTEX];
int     nnbr;

void    splitclassify(SP)
vector  SP;
{
        int     i;

        nedg = 0;
        for(i=0; i<nvtx; i++)
        {
                getneighborhood(soov[i], SP);
                reclassifyonsectors(SP);
                reclassifyonedges();
                insertnulledges();
        }
}
```

Program 14.3 Vertex neighborhood classifier.


```
void    getneighborhood(vtx, SP)
Vertex  *vtx;
vector  SP;
{
        HalfEdge        *he;
        vector          bisect;
        double          d;

        nnbr = 0;
        he = vtx->vedge;
        do
        {
                nbr[nnbr].sector = he;
                d = dist(he->nxt->vtx->vcoord, SP);
                nbr[nnbr++].cl = comp(d, 0.0, EPS);
                if(checkwideness(he) == 1)
                {
                        bisector(he, bisect);
                        nbr[nnbr].sector = he;
                        d = dist(bisect, SP);
                        nbr[nnbr++].cl = comp(d, 0.0, EPS);
                }
        }
        while((he = mate(he)->nxt) != vtx->vedge);
}
```

Program 14.4 Initial classification.

Initial Classification

Let us now elaborate the classification procedures. First, the procedure **getneighborhood** (Program 14.4) gathers the edges of the neighborhood into the array, and computes the initial classification for all edges by measuring the signed distances of the final vertices of each edge of the neighborhood.

For later parts of the algorithm it will be convenient to store "wide" sectors (i.e., sectors whose bounding edges make an angle not less than 180 degrees) as if they were replaced by two "small" ones. Therefore, the procedure constructs the bisector of each such sector, and inserts the sector also with the classification of the bisector. The procedures **checkwideness** and **bisector** for checking the wideness and constructing the bisector for a wide sector are left as exercises to the reader.

Observe that the literals ABOVE, ON, and BELOW were chosen in Program 14.3 so that the result of comp can be used directly as the classification code.

Reclassification Procedures

The next two procedures implement the two reclassification rules stated.

The procedure **reclassifyonsectors** locates all sectors entirely coplanar with *SP*, and reclassifies them as explained above. In Program 14.5, the test for coplanarity is implemented by calculating the cross product of the face normal vector and the normal vector of *SP*, and testing whether the result is a null vector by means of a scalar product. If so, another scalar product tells us whether the normal vectors are actually colinear or opposite.

The final procedure **reclassifyonedges** is relatively straightforward; see Program 14.6. It just scans the neighborhood and applies the second reclassification rule.

14.6 INSERTION OF NULL EDGES

From the final result of the classification, it is easy to read where the vertex neighborhood is intersected by *SP*: if the classifications of two consecutive edges of the cycle differ, the sector in between has an intersection. What remains is the interpretation of this information, and the insertion of the appropriate null edges.

14.6.1 Insertion Rules

The goal of the insertion procedure is to separate all parts of the neighborhood that occur above *SP* from those that occur below it. To illustrate the insertion, let us study the sample case of Figure 14.9. On the left, the results of the classification are indicated. Wide sectors are also indicated. On the right, the expected configuration of null edges is shown. In this case, there are two distinct sequences of ABOVE-edges, and two null edges must be inserted to separate them. To represent the fact that a part of the leftmost wide sector will occur above *SP* even though its bounding edges are below *SP*, a "dangling" null edge must be inserted.

As we have the freedom of choosing an orientation to the null edges, we select to orient them always from the vertex below *SP* to the vertex above *SP*. This property will be very useful to us when joining the null edges to form section polygons.


```
void    reclassifyonsectors(SP)
vector  SP;
{
        Face            *f;
        vector          c;
        double          d;
        int             i;

        for(i=0; i<nnbr; i++)
        {
                f = nbr[i].sector->wloop->lface;
                cross(c, f->feq, SP);
                d = dot(c, c);
                if(comp(d, 0.0, EPS*EPS) == 0)
                {
                        d = dot(f->feq, SP);
                        if(comp(d, 0.0, EPS) == 1)
                        {
                                nbr[i].cl = BELOW;
                                nbr[(i+1)%nnbr].cl = BELOW;
                        }
                        else
                        {
                                nbr[i].cl = ABOVE;
                                nbr[(i+1)%nnbr].cl = ABOVE;
                        }
                }
        }
}
```

Program 14.5 Reclassification of ON-sectors.

```
void    reclassifyonedges()
{
        int             i;

        for(i=0; i<nnbr; i++)
        {
                if(nbr[i].cl == ON)
                {
                        if(nbr[(nnbr+i-1)%nnbr].cl == BELOW)
                        {
                                if(nbr[(i+1)%nnbr].cl == BELOW)
                                        nbr[i].cl = ABOVE;
                                else    nbr[i].cl = BELOW;
                        }
                        else    nbr[i].cl = BELOW;
                }
        }
}
```

Program 14.6 Reclassification of ON-edges.

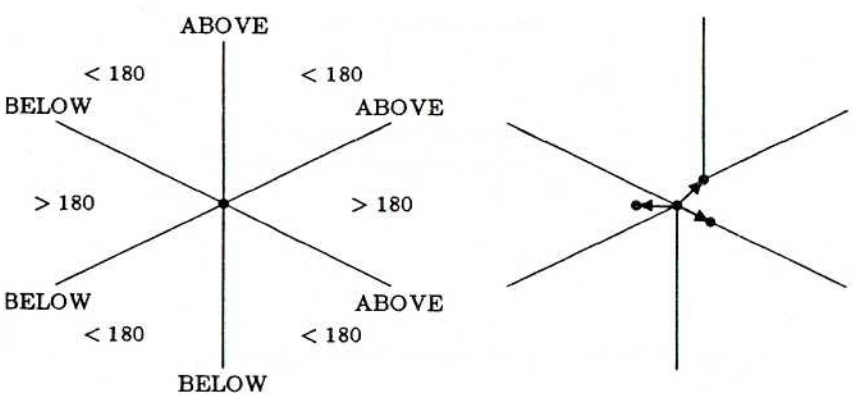


Figure 14.9 Example of the insertion of null edges.

14.6.2 Insertion Procedure

The actual insertion procedure is greatly simplified by the convention of storing "wide" sectors twice in the array representation of the neighborhood. By virtue of this, wide sectors require no special treatment.

In the procedure `insertnulledges` of Program 14.7, the first `while`-loop locates the head of an ABOVE-sequence. If none is found, the algorithm terminates: all edges are on the same side of *SP*. Otherwise, the third `while`-loop locates the final sector of the sequence; thereafter, the null edge can be inserted. Finally, the last `while`-loop locates the start of the next sequence. The second, "perpetual" `while(1)`-loop will be exited when the head of the first sequence is hit again.

If wide sectors are present, the head and the tail of a sequence may in fact be the same sector; in this case, a "dangling" null edge will be inserted, as required. The reader is urged to examine the operation of the insertion algorithm for the sample case of Figure 14.9.

14.7 JOINING OF NULL EDGES

The joining step is supposed to combine the null edges (that mark individual intersection points) created in the previous step together to create null faces (that mark the section polygons). This is accomplished by connecting the end vertices of neighbor null edges with new edges.

14.7.1 Joining Order

To work properly, the joining algorithm must decide which null edges are joined. Fortunately, it can use the topological relationships available in the half-edge data structure to make this decision. Note that except for their geometry, null edges are quite ordinary edges of the half-edge data structure. In particular, they are linked to halfedges, and their neighboring loops and faces can be accessed.

This gives us one way to characterize "neighbor" null edges: they must occur in the same face. We can go further by observing the fact that null edges are always topologically oriented from the vertex below *SP* to the vertex above *SP*. This means that two null edges can be combined only by joining their head vertices with each other with a new edge, and respectively their tail vertices. This is equivalent to the requirement that halves of the null edges must occur in the same face, but in opposite orientations. These tests are implemented in Program 14.8.

Unfortunately, as a face may contain any number of null edges, topological neighborship does not alone determine the proper joining sequence

```
void    insertnulledges()
{
        int             start, i;
        HalfEdge        *head, *tail;

        i = 0;
        while(!(nbr[i].cl == BELOW && nbr[(i+1)%nnbr].cl == ABOVE))
                if(++i == nnbr) return;
        start = i;
        head = nbr[i].sector;
        while(1)
        {
                while(!(nbr[i].cl == ABOVE &&
                        nbr[(i+1)%nnbr].cl == BELOW))
                        i = (i+1) % nnbr;
                tail = nbr[i].sector;
                lmev(head, tail, ++maxv,
                        head->vtx->vcoord[0],
                        head->vtx->vcoord[1],
                        head->vtx->vcoord[2]);
                sone[nedg++] = head->prv->edg;
                while(!(nbr[i].cl == BELOW &&
                        nbr[(i+1)%nnbr].cl == ABOVE))
                {
                        i = (i+1) % nnbr;
                        if(i == start) return;
                }
        }
}
```

Program 14.7 Insertion of null edges.

```
int             neighbor(h1, h2)
HalfEdge        *h1, *h2;
{
        return(h1->wloop->lface == h2->wloop->lface &&
               ((h1 == h1->edg->he1 && h2 == h2->edg->he2) ||
                (h1 == h1->edg->he2 && h2 == h2->edg->he1)));
}
```

Program 14.8 Neighborship test.

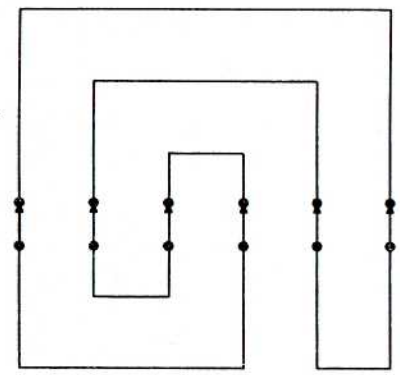


Figure 14.10 Joining example.

uniquely. For a specific example, consider the case of Figure 14.10. The leftmost null edge can be joined topologically with three of the five other null edges marked. The figure also demonstrates that the correct neighbor is *not* necessarily either of the null edges found by scanning the halfedges of the face to the left and to the right of a null edge.

The obvious solution is to pick the *nearest* null edge to a given one for the joining operation. We implement this by considering null edges of *sone* in a lexicographically sorted order along x, y, and z. That is, null edges with at identical x are ordered along y; if even the y-coordinates are equal, z is finally used. Of course, equalities must be determined with comp. Because all null edges lie on *SP*, the lexicographic order together with the topological neighborship defines the correct joining sequence uniquely.

14.7.2 Implementation of the Joining Algorithm

The operation of the joining algorithm consists of processing the null edges in the lexicographic order, and determining whether the next edge can be combined with any yet unfinished section polygon(s). During the lexicographic sweep, not yet finished section polygons are represented in terms of two pointers to halfedges corresponding with the current loose ends of the section polygon.

The detailed operation of the joining algorithm is best explained with help of the Figure 14.11. In this sample case, the block (a) should be split into two smaller ones (b). After the vertex neighborhood classification, we have the arrangement of null edge shown in the plane model (c); dots

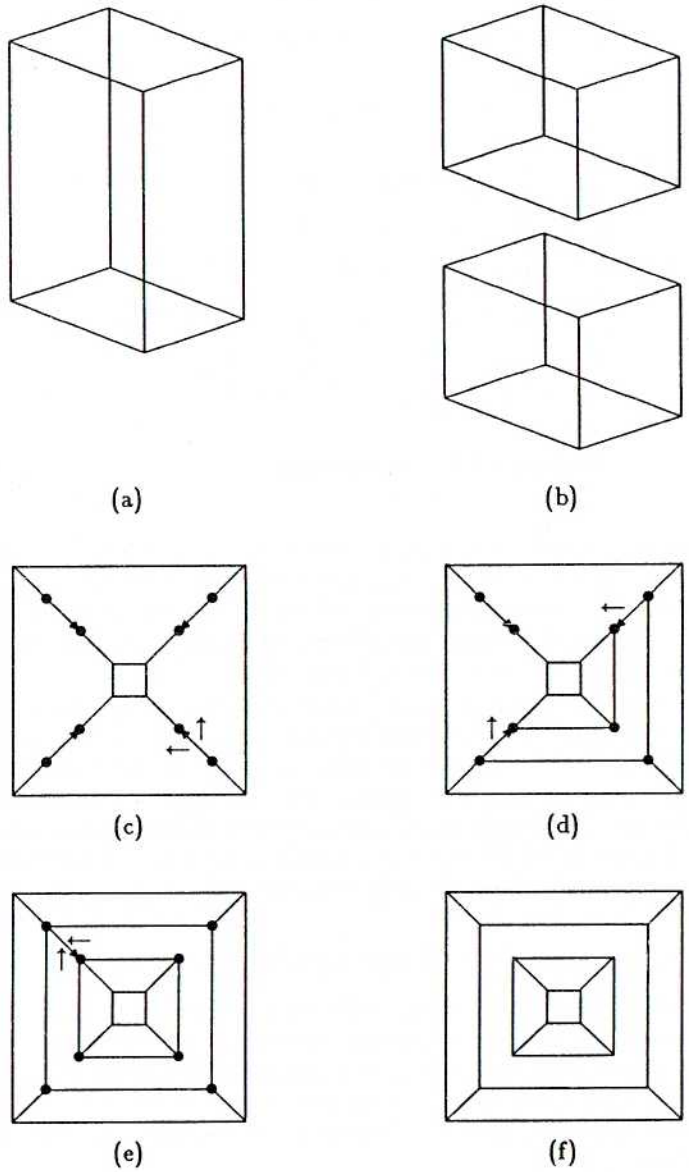


Figure 14.11 Operation of the joining algorithm.

joined with a large arrow denote the null edges and their orientation.

The joining algorithm starts by picking a null edge (actually, the lexicographically smallest one); its two halves form the initial two loose ends of a section polygon. This is indicated with the smaller arrows. When new null edges are added to the section polygon, the ends are updated. Null edges can be removed when they do not form a loose end anymore (see Figure 14.11(d)). Finally, two loose ends may be combined if they can both by joined with a new null edge (e).

The procedure of Program 14.9 implements the joining algorithm. The algorithm begins by sorting *sone* lexicographically by x, y, and z by a procedure **sortnulledges**. Thereafter, it retrieves each null edge in their sorted order by means of the procedure **getnextnulledge**, and performs the appropriate operations on them. The procedures **sortnulledges** and **getnextnulledge** are not elaborated.

In the general situation, some loose ends will already exist. By means of the procedure **canjoin**, we learn for both halves of the next edge whether the half can be joined with a loose end. If so, the actual joining is performed. After joining, the edge of the loose end can be cut, if the other half of the edge is not loose. The procedure **isloose** is assumed to determine whether its argument halfedge is a loose end.

The set of loose ends is maintained by the procedure **canjoin**. If a current loose end can be joined, it can be removed immediately from the set, because a joining operation will occur. Otherwise, the nonmatching half of the new null edge itself becomes a loose end.

The remaining pieces of the joining algorithm are **join**, for combining two null edges, and **cut**, for removing a null edge after it has been combined. These procedures can readily be implemented with Euler operators as in Program 14.10.

Procedure **join** basically has to determine whether the two null edges occur in the same loop in order to decide which Euler operators to use for combining null edges. Care must be taken to make sure that potential internal loops end up in correct faces after a MEF. The procedure **laringmv** fills this need. To make **join** directly useful for the purposes of the next chapter, the **lkef**'s are guarded by **if**-statements that check that the edges are not already connected with each other.

The completion of a section polygon is detected in procedure **cut**: when the same loop appears on both sides of a null edge to be removed, it must be the last one remaining in that section polygon (see Figure 14.11(e,f)), and **cut** can store a pointer to the face into *sonf*.

Figure 14.12 gives an example of the operation of the joining algorithm. The eight null edges are sorted here along x and z; they all have the equal

```
static HalfEdge *ends[30];
static int      nend;

void    splitconnect()
{
        Edge            *nextedge, *getnextnulledge();
        HalfEdge        *h1, *h2, *canjoin();

        nfac = 0;
        sortnulledges();
        while(nextedge = getnextnulledge())
        {
                if(h1 = canjoin(nextedge->he1))
                {
                        join(h1, nextedge->he1);
                        if(!isloose(mate(h1))) cut(h1);
                }
                if(h2 = canjoin(nextedge->he2))
                {
                        join(h2, nextedge->he2);
                        if(!isloose(mate(h2))) cut(h2);
                }
                if(h1 && h2) cut(nextedge->he1);
        }
}
HalfEdge        *canjoin(he)
HalfEdge        *he;
{
        HalfEdge        *ret;
        int             i, j;

        for(i = 0; i<nend; i++)
                if(neighbor(he, ends[i]))
                {
                        ret = ends[i];
                        for(j=i+1; j<nend; j++)
                                ends[j-1] = ends[j];
                        nend--;
                        return(ret);
                }
        ends[nend++] = he;
        return((HalfEdge *) NIL);
}
```

Program 14.9 The joining algorithm.


```
void            join(h1, h2)
HalfEdge        *h1, *h2;
{
        Face    *oldf, *newf;

        oldf = h1->wloop->lface;
        newf = (Face *) NIL;
        if(h1->wloop == h2->wloop)
        {
                if(h1->prv->prv != h2)
                        newf = lmef(h1, h2->nxt, ++maxf);
        }
        else    lmekr(h1, h2->nxt);
        if(h1->nxt->nxt != h2)
        {
                lmef(h2, h1->nxt, ++maxf);
                if(newf && oldf->floops->next1)
                        laringmv(oldf, newf);
        }
}

void            cut(he)
HalfEdge        *he;
{
        if(he->edg->he1->wloop == he->edg->he2->wloop)
        {
                sonf[nfac++] = he->wloop->lface;
                lkemr(he->edg->he1, he->edg->he2);
        }
        else    lkef(he->edg->he1, he->edg->he2);
}
```

Program 14.10 Joining and cutting of null edges.

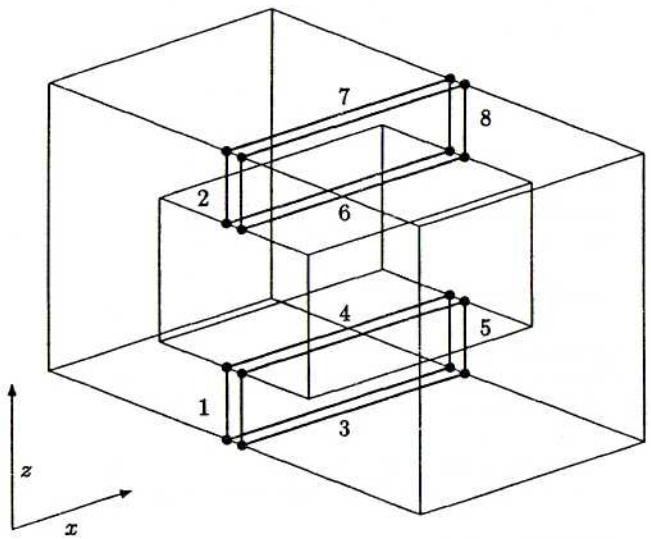


Figure 14.12 Example of the joining algorithm.

y-coordinate. The numbers 1 ... 8 indicate the joining order of null edges. Note that in this case two section polygons are processed simultaneously.

14.8 CLASSIFICATION OF FACES

The final missing part of the splitting algorithm is the actual division of the sectioned solid into two solids representing the results. Fortunately, the joining algorithm supplies useful information for completing the remaining task.

Recall that the joining algorithm inserts a "null face" along the section polygon. The null face will contain two loops running through pairwise coincident vertices. Also, recall that null edges were generated so that they always point "up," i.e. so that their starting vertex belongs to the part below the splitting plane *SP*, and their final vertex similarly to the part above *SP*. By examining the joining algorithm (particularly, procedures **neighbor**, Program 14.8 and **join**, Program 14.10) one can see that "down" vertices are always connected with "down" vertices, and similarly "up" vertices with "up" vertices. Therefore, we know that the "inner" loop should appear in the part *Above*, and the "outer" loop in the part *Below*.


```
void      splitfinish(S, Above, Below)
Solid     *S, **Above, **Below;
{
     int        i;

     for(i=0; i<nfac; i++)
          sonf[nfac+i] = lmfkrh(sonf[i]->floops->nextl, ++maxf);
     *Above = (Solid *) new(NIL, SOLID);
     *Below = (Solid *) new(NIL, SOLID);
     classify(S, *Above, *Below);
     cleanup(*Above);
     cleanup(*Below);
}
```

Program 14.11 Generation of results.

This makes the actual separation of the resulting solids straightforward (see Program 14.11). First, the null faces are transformed to two section faces by means of a MFKRH. This effectively increases the connectivity of the half-edge data structure so that it now models a disconnected solid.

The heart of the final phase is the classification of the faces of the now disconnected original solid into the solids *Above* and *Below*, the task of procedure **classify**. Observing that of all section faces, we already know where they should go, and that all other faces can be classified according to their topological neighborhood relationships to faces already classified, **classify** is easily implemented as a recursive depth-first-search algorithm that starts from the section faces and recursively moves all neighbors of a face that have not yet been moved from *S* to *Above* or *Below*. The implementation of Program 14.12 uses procedures **addlist** and **dellist** described in Chapter 11 to manipulate the list of faces of a solid directly.

Note that **classify** only constructs the lists of faces of *Above* and *Below*—their lists of edges and vertices remain empty. The procedure **cleanup** is assumed to reconstruct these lists based on the list of faces. Its implementation is straightforward and is left to the reader.

14.9 SLICING

The splitting algorithm described above works on the half-edge data structure of a solid, and modifies it directly in order to create the results. Hence the original object disappears. A simple variation of the basic algorithm

```
void    classify(S, Above, Below)
Solid   *S, *Above, *Below;
{
        int             i;

        for(i = 0; i < nfac; i++)
        {
                movefac(sonf[i], Above);
                movefac(sonf[nfac+i], Below);
        }
}

void    movefac(f, s)
Face    *f;
Solid   *s;
{
        Loop            *l;
        HalfEdge        *he;
        Face            *f2;

        dellist(FACE, f, f->fsolid);
        addlist(FACE, f, s);
        l = f->floops;
        while(l)
        {
                he = l->ledg;
                do
                {
                        f2 = mate(he)->wloop->lface;
                        if(f2->fsolid != s)
                                movefac(f2, s);
                }
                while((he = he->nxt) != l->ledg);
                l = l->nextl;
        }
}
```

Program 14.12 Classification of faces.

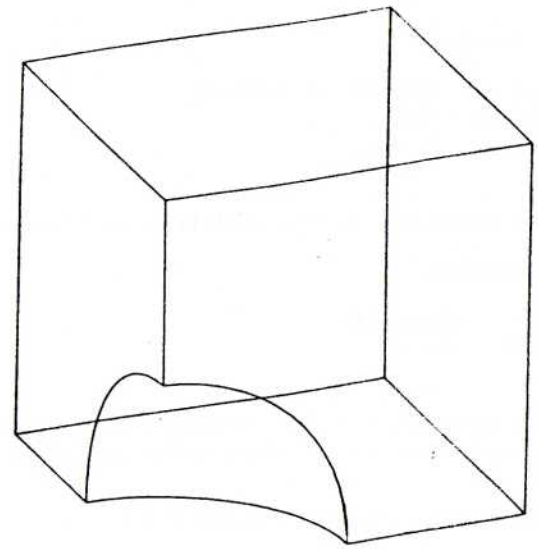


Figure 14.13 A slicing operation.

generates the section polygons only and leaves the original object intact. We call this operation *slicing*.

Slices are useful for visualizing complicated objects. Sometimes they can be used also for modeling purposes, say, for the design of a container for a given part. For an example, see Figure 14.13.

To modify the splitting algorithm to a slicing algorithm, only modest changes in the joining algorithm and the later steps of the splitting algorithm are necessary. In particular, the procedure `join` that builds section polygons can be modified to build the polygons as a new solid *slice*, instead of manipulating the solid itself. To nullify the effects of the algorithm to the original solid, all new vertices created by it should be deleted.

BIBLIOGRAPHIC NOTES

Figures 14.2, 14.3, 14.4, 14.5, and 14.6 have been previously published in the article "Boolean operations of 2-manifolds through vertex neighborhood classification" by M. Mäntylä in *Transactions on Graphics*, Volume 5, Number 1. ©1986, Association for Computing Machinery, Inc. Reprinted by permission.

PROBLEMS

14.1. Implement the procedure

```
void      bisector(he, bisect)
HalfEdge  *he;
vector    bisect;
```

that constructs a vector along the bisector of the sector defined by *he*.

14.2. Implement the procedure

```
void    cleanup(s)
Solid   *s;
```

that reconstructs the lists of edges and vertices of its argument solid so that they will contain those and only those entities adjacent to the faces of the solid.

Hint: Read again last the paragraph of Section 10.4.1.

14.3. Outline the implementation of a slicing procedure based on the splitting algorithm.